# INFORME DE LABORATORIO

| INFORMACIÓN BÁSICA | | | | | |
|---|---|---|---|---|---|
| **ASIGNATURA:** | Tecnología de Objetos | | | | |
| **TÍTULO DE LA PRÁCTICA:** | C++ Herencia, Polimorfismo, Qt | | | | |
| **NÚMERO DE PRÁCTICA:** | 03 | **AÑO LECTIVO:** | 2026-B | **NRO. SEMESTRE:** | VI (sexto) |
| **FECHA DE PRESENTACIÓN** | 18/09/2026 | **HORA DE PRESENTACIÓN** | 23:59 | | |
| **INTEGRANTE (s):**<br>Franco Jesus Cahua Soto | | | | **NOTA:** | |
| **DOCENTE(s):** Mg. Ing. Rene Alonso Nieto Valencia | | | | | |
| Modalidad: Individual | | Inicio de la práctica: 14/09/2026 | | | |

## SOLUCIÓN Y RESULTADOS

### I. INTRODUCCIÓN

Tras consolidar en la práctica anterior los fundamentos de la encapsulación y la administración explícita de la memoria mediante punteros, la presente sesión de laboratorio aborda los dos mecanismos que permiten escalar el modelo de objetos hacia sistemas de gran envergadura: la **herencia** y el **polimorfismo**.

Ambos conceptos constituyen el eje sobre el que se sostiene la reutilización de código a escala industrial. La herencia establece relaciones jerárquicas del tipo *es un* (IS-A) que permiten modelar clasificaciones naturales del dominio, distribuyendo el comportamiento común en clases base. El polimorfismo, por su parte, garantiza que una misma interfaz exprese comportamientos distintos según el tipo dinámico real del objeto, habilitando la extensión de sistemas cerrados sin modificar el código ya validado.

En la presente práctica de laboratorio **N.° 03** de la asignatura **Tecnología de Objetos**, se desarrolla la herencia simple modelando el oficio como criterio de clasificación de las personas; se extiende el modelo mediante **herencia múltiple** para representar los fenotipos combinados de las cepas del coronavirus, resolviendo el *problema del diamante* con herencia virtual; se implementa un **Árbol de Expresiones Binario** (*Binary Expression Tree*) que resuelve operaciones de suma y multiplicación empleando clases y punteros; y se analizan los **punteros inteligentes** de C++11 como evolución del modelo manual de propiedad del lenguaje.

# SOLUCIÓN Y RESULTADOS

## I.1 Objetivos de la práctica

- Comprender el mecanismo de herencia en C++ como mecanismo de reutilización de código y clasificación jerárquica del dominio.
- Diferenciar los niveles de acceso `public`, `protected` y `private` aplicados tanto a los miembros como al propio acto de heredar.
- Implementar jerarquías de herencia simple con clases base abstractas y funciones virtuales puras como contrato de interfaz.
- Modelar la herencia múltiple mediante la composición de fenotipos ortogonales y resolver el rombo con herencia virtual.
- Aplicar RTTI con `dynamic_cast` para identificar en tiempo de ejecución el tipo dinámico real de un objeto.
- Implementar un Árbol de Expresiones Binario con clases y punteros capaz de evaluar operaciones de suma y multiplicación respetando la precedencia aritmética.
- Analizar los punteros inteligentes de C++11 (`unique_ptr`, `shared_ptr`, `weak_ptr`) y su impacto en el manejo de la propiedad de recursos.
- Diferenciar técnicamente la sobrecarga de operadores y funciones respecto del polimorfismo de tipo dinámico.

## II. MARCO CONCEPTUAL

### II.1 Herencia: clasificación jerárquica del dominio

La herencia es el proceso mediante el cual un objeto puede adquirir las propiedades y el comportamiento de otro. Su ventaja operativa reside en la **reutilización de código**: los atributos y métodos comunes se declaran una sola vez en la clase base (*superclase*) y las clases derivadas (*subclases*) los especializan.

**Listado 1:** Sintaxis de herencia simple y niveles de acceso

```
class Derivada : public Base { ... };    // la clase base queda COMO PARTE PUBLICA
class Privada : private Base { ... };    // solo la derivada accede a la base
class Proteg : protected Base { ... };   // acceso medio: las derivadas de Proteg
```

La clasificación jerárquica resulta indispensable cuando las entidades del dominio no solo comparten atributos, sino que además se relacionan entre sí por una pertenencia taxonómica. Un *bombero* es una *persona* que trabaja; un *arquitecto* es una *persona* que proyecta. Modelar cada oficio desde cero duplicaría seis campos idénticos; modelarlos mediante herencia los declara una única vez en la clase base `Persona`.

## SOLUCIÓN Y RESULTADOS

### II.2 Niveles de acceso en la herencia

La palabra clave que acompaña al nombre de la clase base en la lista de herencia controla la *regla de visibilidad* del artefacto heredado:

| Nivel | Para la derivada | Efecto sobre la interfaz pública |
|---|---|---|
| `public` | Acceso total a los miembros públicos y protegidos de la base. | La interfaz de la base queda disponible públicamente en la derivada: es el nivel habitual de la herencia simple. |
| `protected` | Acceso total, pero la interfaz pública de la base se vuelve inaccesible. | La clase derivada sigue reconociendo `Persona`, pero un usuario externo ya no puede invocar sus métodos públicos. |
| `private` | Acceso total únicamente dentro de la propia clase derivada. | La herencia se reduce al detalle de implementación; la relación *es un* queda oculta y debe delegarse explícitamente. |

### II.3 Clases abstractas y funciones virtuales puras

Una clase que contiene al menos una función virtual pura (`= 0`) se denomina **abstracta** y no puede instanciarse. Este recurso expresa un **contrato**: la clase base declara qué comportamiento debe existir, sin imponer cómo se implementa. La clase derivada queda obligada a redefinirlo, garantizando la completitud de la jerarquía.

**Listado 2:** Interfaz abstracta con funcion virtual pura (ejemplo ilustrativo)

```
class Figura {
protected:
    double base, altura;
public:
    Figura(double b, double h) : base(b), altura(h) {}
    virtual ~Figura() = default;                  // OBLIGATORIO: destructor virtual
    virtual double area() const = 0;             // interfaz pura
    virtual std::string nombre() const = 0;
};
```

La declaración de un **destructor virtual** en la clase base resulta crítica. Sin ella, la destrucción de un objeto derivado a través de un puntero a la clase base ejecutaría únicamente el destructor base, omitiendo la liberación de los recursos propios de la derivada y provocando una fuga de memoria silenciosa.

### II.4 Polimorfismo y enlace dinámico

El polimorfismo es la cualidad que permite que un mismo nombre designe dos o más propósitos relacionados pero técnicamente diferentes. En C++ se distinguen dos tipos de enlace en el momento de la invocación, según el criterio que el compilador emplea para seleccionar la implementación:

- **Enlace estático (tiempo de compilación):** se resuelve examinando el *tipo estático* declarado de la expresión. Es el mecanismo de la **sobrecarga** (*overloading*): múltiples funciones comparten nombre pero difieren en la lista de parámetros.

## SOLUCIÓN Y RESULTADOS

- **Enlace dinámico (tiempo de ejecución):** se resuelve examinando el *tipo dinámico* real del objeto. Es el mecanismo del **polimorfismo**: una única declaración en la clase base se despacha a la redefinición concreta que posea el objeto en memoria.

Cuando una clase contiene al menos una función virtual, el compilador genera una **tabla de funciones virtuales** (*vtable*) asociada a cada tipo, y cada instancia almacena un puntero oculto (`vptr`) que la referencia. La invocación polimórfica se resuelve en dos pasos: el `vptr` del objeto localiza su *vtable* y desde ella se obtiene la dirección de la función efectiva.

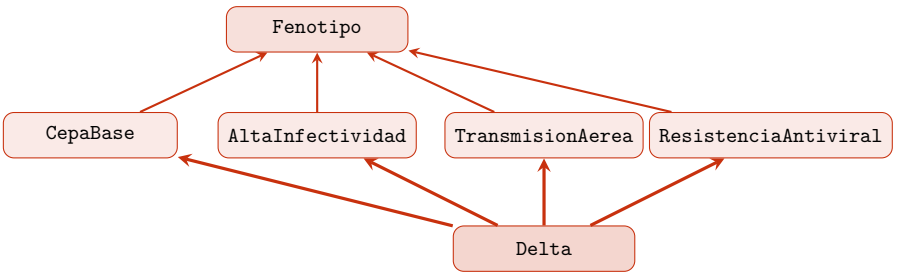
**Listado 3:** Polimorfismo: una sola llamada, N comportamientos (ejemplo ilustrativo)

```
std::vector<Figura*> figuras;                  // todas "ven" como Figura*
for (Figura* f : figuras) {
    std::cout << f->nombre() << " = " << f->area() << "\n";
}   // cada objeto responde con SU propia implementacion
```

### II.5 Herencia múltiple y el problema del diamante

La herencia múltiple permite que una clase derivada herede simultáneamente de más de una clase base, combinando funcionalidades que no guardan jerarquía entre sí. El modelo de *mixins* (componentes ortogonales de comportamiento) es su aplicación más extendida en la ingeniería de software real.

Sin embargo, la herencia múltiple genera el **problema del diamante**: cuando dos rutas de herencia convergen en un ancestro común, dicho ancestro se duplica en la memoria y su interfaz resulta **ambigua**.




La **herencia virtual** (`public virtual`) instruye al compilador para que conserve una única instancia del ancestro común compartida por todas las rutas:

**Listado 4:** Resolución del diamante con herencia virtual

```
class CepaBase : public virtual Fenotipo { ... };   // Fenotipo se declara UNA vez
class TransmisionAerea : public virtual Fenotipo { ... };
class ResistenciaAntiviral : public virtual Fenotipo { ... };
// Sin `virtual` el compilador reportaria:
//   error: base class 'Fenotipo' is ambiguous
```

## SOLUCIÓN Y RESULTADOS

### II.6 Identificación de tipos en tiempo de ejecución (RTTI)

Cuando una variable se declara como puntero a la clase base pero el objeto real pertenece a una derivada, el lenguaje ofrece `dynamic_cast` para preguntar por el tipo dinámico efectivo. Esta conversión se realiza de forma segura: devuelve `nullptr` si la jerarquía no admite la relación, en lugar de producir una conversión inválida.

**Listado 5:** Consulta de fenotipos mediante dynamic$_cast(ejemploilustrativo)$

```
CepaBase* ptr = new Omicron();
if (dynamic_cast<ResistenciaAntiviral*>(ptr) != nullptr) {
    std::cout << "La variante presenta resistencia antiviral.\n";
}
```

### II.7 Entorno de desarrollo Qt

Qt es un framework multiplataforma de interfaces gráficas que modela la aplicación como un grafo de objetos declarativo. Sus componentes fundamentales son:

- **QWidgets:** conjunto de controles de interfaz (botones, etiquetas, cuadros de texto) organizados en jerarquías de contenedores.
- **Señales y slots:** mecanismo de comunicación basada en eventos. Un objeto emite una *señal* (`emit`) cuando ocurre un cambio de estado; otro objeto conectado mediante `connect()` reacciona ejecutando su *slot*.
- **MOC (Meta-Object Compiler):** herramienta de Qt que procesa las clases que emplean signals/slots, generando código C++ adicional (`moc_*.cpp`) que la biblioteca estándar no reconoce.
- **Qt Creator:** IDE que asiste la construcción de proyectos mediante el asistente *File | New File or Project | Application | Qt Widgets Application*, generando automáticamente la estructura del proyecto, el `.pro` y el `.ui`.

## III. ANÁLISIS DE EJERCICIOS RESUELTOS POR EL DOCENTE

### III.1 Herencia entre niños, jóvenes y adultos

Se desarrolló una jerarquía de tres niveles que clasifica a las personas por etapa evolutiva, cada una con habilidades propias. La clase base `Persona` concentraba los atributos comunes (nombre, edad, documento de identidad), mientras que las derivadas `Nino`, `Joven` y `Adulto` incorporaban las habilidades específicas de su tramo etario. Se mostró que el constructor de la derivada invocaba primero al de la base, garantizando la correcta inicialización de la parte común antes de los atributos propios.

## SOLUCIÓN Y RESULTADOS

### III.2 Polimorfismo entre mamíferos con función virtual

Se modeló la relación jerárquica de los mamíferos mediante una clase abstracta `Mamifero` con la función virtual pura `sonido()`. Cada derivada (`Perro`, `Gato`, `Vaca`) redefinió dicho método. Al invocar `sonido()` sobre un vector de punteros a `Mamifero`, cada objeto emitía su sonido característico, demostrando el despacho dinámico característico del polimorfismo.

### III.3 Conociendo el entorno de Qt Creator

Se recorrió el flujo de creación de una aplicación de escritorio: *File | New File or Project | Application | Qt Widgets Application | Choose*. Qt Creator genera automáticamente la estructura del proyecto, el archivo de interfaz `.ui` en modo Designer y el `MainWindow` con su clase `QtWidgets`, ilustrando la separación entre la lógica de negocio y la presentación gráfica.

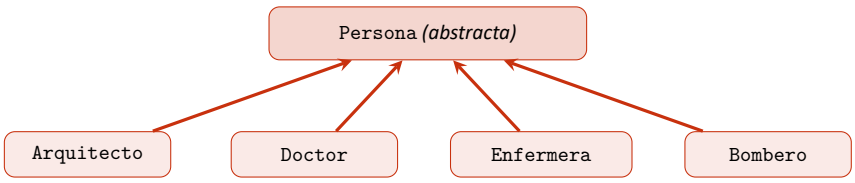
## IV. DESARROLLO DE EJERCICIOS PROPUESTOS

A continuación se detalla la solución completa a los 3 ejercicios propuestos, acompañada de los fundamentos técnicos, diagramas de clases, códigos C++ verificados y evidencia de ejecución.

### IV.1 Ejercicio 1: Herencia de Personas por Oficio

**Enunciado:** Crear en C++ la herencia de diferentes personas por oficio (arquitecto, doctor, enfermera, bombero).

**Diseño de la solución:** Se construyó una jerarquía de **herencia simple** con la clase base abstracta `Persona` y cuatro clases derivadas de herencia pública. Los atributos comunes (`nombre`, `apellido`, `edad`, `documento`, `aniosExperiencia`, `salarioBase`) se declaran `protected` para que las derivadas accedan directamente sin exponerlos al usuario externo. La interfaz se define mediante tres funciones virtuales puras (`obtenerOficio`, `tareaPrincipal`, `calcularSalarioMensual`) y un método virtual con implementación por defecto (`presentarHabilidad`).




**Listado 6:** Clase base abstracta y herencia publica en ejercicio1.cpp

```
class Persona {
protected:
    std::string nombre;
    std::string apellido;
    int edad;
    std::string documento;
```

## SOLUCIÓN Y RESULTADOS

```
    int aniosExperiencia;
    double salarioBase;

public:
    Persona(const std::string& nom, const std::string& ape, int ed,
            const std::string& doc, int anios, double salario)
        : nombre(nom), apellido(ape), edad(ed), documento(doc),
          aniosExperiencia(anios), salarioBase(salario) {}

    // Destructor virtual: imprescindible para destruir correctamente los
    // objetos proprios de las clases derivadas a traves de un puntero base.
    virtual ~Persona() = default;

    // --- INTERFAZ PURAMENTE VIRTUAL: obliga a toda derivada a implementarla ---
    virtual std::string obtenerOficio() const = 0;
    virtual std::string tareaPrincipal() const = 0;
    virtual double calcularSalarioMensual() const = 0;

class Arquitecto : public Persona {
private:
    std::string especialidad;
    std::string colegiatura;
    int proyectosTerminados;

public:
    Arquitecto(const std::string& nom, const std::string& ape, int ed,
               const std::string& doc, int anios, double salario,
               const std::string& esp, const std::string& coleg, int proyectos)
        : Persona(nom, ape, ed, doc, anios, salario),
          especialidad(esp), colegiatura(coleg), proyectosTerminados(proyectos) {}

    std::string obtenerOficio() const override { return "ARQUITECTO"; }

    double calcularSalarioMensual() const override {
        return salarioBase + (proyectosTerminados * 350.0) + (aniosExperiencia * 25.0);
    }
```

Se demostró además el efecto del nivel de acceso `private` mediante la clase `AsistenteTecnico`, que hereda privadamente de `Arquitecto`: toda la interfaz de `Arquitecto` queda oculta y debe re-exponerse de forma explícita, como hace `getOficioDelegado()`, que delega en `obtenerOficio()`.

**Evidencia de ejecución: la misma llamada produce cuatro comportamientos distintos**

```
>>> 2. ITERACION POLIMORFICA (misma llamada, 4 cuerpos):
  [ARQUITECTO] Lucia Fernandez Ramirez | Arq. Sustentable | SIC-2011-4478
      => Actividad: Elabora planos y dirige obras de construccion
      => Proyectos terminados: 28
  [DOCTOR] Andres Quispe Loayza | Cardiologia | RNE-8871-22
      => Actividad: Diagnostica, receta y hace seguimiento clinico
      => Pacientes atendidos: 145
  [ENFERMERA] Rosa Quispe Mamani | Enf. Intensiva | Turno Nocturno
      => Actividad: Administra medicamentos y toma signos vitales
```

## SOLUCIÓN Y RESULTADOS

```
      => Pacientes a cargo: 6
  [BOMBERO] Carlos Ccahuana Tito | Estacion Central 104 | Brigadista PARAM.
      => Actividad: Extingue incendios y realiza rescates
      => Incidentes atendidos: 67

>>> 3. POLIMORFISMOS EN EL CALCULO DEL SUELDO (misma interfaz):
  OFICIO          EDAD    EXPERIENCIA    SUELDO MENSUAL
  ------------------------------------------------------
  ARQUITECTO      41      15             S/.    13675.00
  DOCTOR          47      19             S/.    11865.00
  ENFERMERA       33      8              S/.     2770.00
  BOMBERO         39      13             S/.     8480.00
```

### IV.2 Ejercicio 2: Herencia Múltiple de las Cepas del Coronavirus

**Enunciado:** Del ejemplo anterior, implementar en C++ la herencia múltiple de las cepas del coronavirus.

**Diseño de la solución:** Se extendió el patrón de clasificación taxonómica del ejercicio 1 modelando las variantes virales. Se definieron cuatro componentes ortogonales —la identidad viral (`CepaBase`) y tres *mixins* de comportamiento (`AltaInfectividad`, `TransmisionAerea`, `ResistenciaAntiviral`)— que comparten una raíz común `Fenotipo` declarada como **herencia virtual** para eliminar la ambigüedad del diamante. Cada variante (`Alfa`, `Lambda`, `Delta`, `Omicron`) hereda simultáneamente de tres o cuatro bases, componiendo su fenotipo por mezcla.

**Listado 7:** Composicion de fenotipos por herencia multiple en ejercicio2.cpp

```cpp
class Omicron : public CepaBase, public AltaInfectividad,
                public TransmisionAerea, public ResistenciaAntiviral {
public:
    Omicron() : Fenotipo("OMICRON-MULTI"),
                CepaBase("SARS-CoV-2 Omicron B.1.529", "OMICRON", "Sudafrica", 3, 0.30) {
        setAfinidadSpike(4.10);
        setR0(3.50);
        setPrincipalVecto("aerosol - alta capacidad");
        setResistencia(80.0, 65.0);
    }

    std::string genotipado() const override {
        return "50+ mutaciones en spike - recombinacion";
    }

    std::string resumenFenotipo() const override {
        return "Infectividad ALTA + Transmision AEREA + Antiviral";
    }

    void mostrarFicha() const override {
        CepaBase::mostrarFicha();
        std::cout << "  Genotipo     : " << genotipado() << "\n"
                  << "  Severidad    : " << nivelSeveridad() << "\n";
        mostrarInfectividad();
        mostrarTransmision();
```

## SOLUCIÓN Y RESULTADOS

```
        mostrarAntiviral();
    }
```

Para evitar la duplicación de código entre `Delta` y `Omicron`, se empleó la **composición explícita de mixins**: cada mixin expone su propio método de presentación (`mostrarInfectividad()`, `mostrarTransmision()`, `mostrarAntiviral()`) y la variante concreta decide a cuáles invocar desde su `mostrarFicha()` reimplementado. Este es justamente el consejo práctico de la ingeniería de software ante la herencia múltiple: *prefiera la composición (mixins con métodos explícitos) sobre la herencia múltiple de implementación compartida*.

**Listado 8:** Detección de fenotipos en tiempo de ejecución con RTTI

```cpp
for (CepaBase* v : variantes) {
    if (dynamic_cast<AltaInfectividad*>(v) != nullptr)
        std::cout << "Infectividad ";
    if (dynamic_cast<TransmisionAerea*>(v) != nullptr)
        std::cout << "Transmision ";
    if (dynamic_cast<ResistenciaAntiviral*>(v) != nullptr)
        std::cout << "Antiviral ";
}
```

**Evidencia de ejecución: fichas con composición fenotípica por mixin**

```
[DELTA] CepaBase + 3 mix-ins (rombo completo)
  Codigo         : DELTA-MULTI
  Linaje         : DELTA
  Denominacion   : SARS-CoV-2 Delta B.1.617.2
  Origen         : India
  Incubacion     : 4 dias
  Letalidad IFR : 3.20 %
  Fenotipo       : Infectividad ALTA + Transmision AEREA + Antiviral
  Genotipo      : L452R + T478K - escape a anticuerpos
  Severidad     : CRITICA
    [MIX-IN 1] Afinidad spike/ACE2 = 2.30
    [MIX-IN 2] R0 = 2.20  via aerosol + eventos masivos
    [MIX-IN 3] Remdesivir -35% sensibilidad  escape vacunal 40%

>>> 3. RTTI: IDENTIFICACION DE FENOTIPOS (dynamic_cast)
  ORIGINAL   -> | raiz: ORIGINAL-SIMPLE
  ALFA       -> Infectividad Transmision | raiz: ALFA-MULTI
  LAMBDA     -> Transmision Antiviral | raiz: LAMBDA-MULTI
  DELTA      -> Infectividad Transmision Antiviral | raiz: DELTA-MULTI
  OMICRON    -> Infectividad Transmision Antiviral | raiz: OMICRON-MULTI
```

## SOLUCIÓN Y RESULTADOS

### IV.3 Ejercicio 3: Árbol de Expresiones Binario (suma y multiplicación)

**Enunciado:** Implementar en C++ un Binary Expression Tree que pueda resolver operaciones matemáticas suma y multiplicación (usar clases y punteros).

**Diseño de la solución:** Se implementó un **Árbol de Expresiones Binario** (*Binary Expression Tree*, BET), la estructura de datos clásica de los evaluadores de expresiones. La jerarquía se apoya en la clase base abstracta `Nodo`, cuya interfaz pura delega en las derivadas la evaluación y los tres recorridos; `Hoja` almacena un literal y `NodoOperacion` referencia sus dos hijos mediante punteros *owning* y les aplica el operador. El proceso consta de tres etapas: (1) conversión infija $\rightarrow$ postfija mediante el algoritmo *Shunting-Yard*; (2) construcción del árbol apilando los tokens postfijos; (3) evaluación polimórfica invocando `evaluar()` sobre la raíz, que se despacha de forma recursiva hasta las hojas.

**Listado 9:** Interfaz abstracta del arbol de expresiones en ejercicio3.cpp

```
class Nodo {
public:
    virtual ~Nodo() = default;

    // --- INTERFAZ PURAMENTE VIRTUAL DEL ARBOL DE EXPRESIONES ---
    virtual double evaluar() const = 0;
    virtual bool esHoja() const = 0;
    virtual char simbolo() const = 0;
    virtual void imprimirInorden(std::ostream& out, bool paren) const = 0;
    virtual void imprimirPreorden(std::ostream& out) const = 0;
    virtual void imprimirPostorden(std::ostream& out) const = 0;

    // --- METODOS VIRTUALES CON IMPLEMENTACION POR DEFECTO ---
    virtual int contarNodos() const { return 1; }
    virtual int altura() const { return 1; }

    // --- METODO VIRTUAL PURO: liberacion recursiva del subarbol ---
    virtual void liberar() = 0;

    // --- Tabla de precedencia aritmetica compartida por toda la jerarquia ---
    static int precedencia(char op) {
        if (op == '*') return 2;
        if (op == '+') return 1;
```

**Listado 10:** Nodo interno con evaluacion polimorfica en ejercicio3.cpp

```
private:
    char operador;
    Nodo* izq;
    Nodo* der;

public:
    NodoOperacion(char op, Nodo* izquierdo, Nodo* derecho)
        : operador(op), izq(izquierdo), der(derecho) {}

    // Nucleo del arbol de expresiones: aplica la operacion a ambos hijos.
    double evaluar() const override {
```

## SOLUCIÓN Y RESULTADOS

```
        const double a = izq->evaluar();
        const double b = der->evaluar();
        switch (operador) {
            case '+': return a + b;
            case '*': return a * b;
            default:
                std::cerr << "ERROR: operador no soportado '" << operador << "'\n";
                return std::numeric_limits<double>::quiet_NaN();
        }
    }
```

El constructor del árbol emplea el algoritmo *Shunting-Yard* con precedencia $* > +$, parentizado mediante una pila de marcadores. Al construir el árbol desde la forma postfija, se garantiza la asociatividad izquierda a derecha.

**Listado 11:** Conversión infija a postfija con precedencia de operadores

```
while (!pila.empty() && pila.back() != "(" &&
       precedencia(pila.back()) >= precedencia(std::string(1, c))) {
    salida.push_back(pila.back());
    pila.pop();
}
pila.push_back(std::string(1, c));
```

**Evidencia de ejecución: conversión, construcción y evaluación con verificación cruzada**

```
>>> 1. CONVERSION INFIJA -> POSTFIJA (algoritmo Shunting-Yard)
  3+4*5              -> 3 4 5 * +
  2*3+4*5            -> 2 3 * 4 5 * +
  (3+4)*5            -> 3 4 + 5 *
  2*(3+4)*(5+6)      -> 2 3 4 + * 5 6 + *
  1+2*3+4*5*6        -> 1 2 3 * + 4 5 * 6 * +

  Expresion infija : 2*(3+4)*(5+6)
  Prefijo  (raiz)  : (* (* 2 (+ 3 4)) (+ 5 6))
  Infijo  (legible): (2 * (3 + 4) * (5 + 6))
  Postfijo (RPN)   : 2 3 4 + * 5 6 + *
  Nodos / Altura   : 9 nodos, 4 niveles
  EVALUACION       : 154.0000
  Referencia       : 154.0000   [COINCIDE]

>>> 3. PROPIEDAD DISTRIBUTIVA: (a+b)*(c+d) = ac+ad+bc+bd
  Forma factorizada : ((2 + 3) * (4 + 5)) = 45.0000
  Forma expandida   : (2 * 4 + 2 * 5 + 3 * 4 + 3 * 5) = 45.0000
  Nodos (factorizada / expandida): 7 / 15
  Conclusion: mismo valor, distinta complejidad estructural.
```

La propiedad distributiva se verificó programáticamente: la forma factorizada $(2+3)\times(4+5)$ produce $45$ con $7$ nodos, mientras que la forma expandida $2\times4+2\times5+3\times4+3\times5$ produce el idéntico resultado con

## SOLUCIÓN Y RESULTADOS

15 nodos. La evaluación se contrastó contra un evaluador iterativo de referencia (postfijo con pila de reales), obteniéndose `COINCIDE` en la totalidad de los casos de prueba.

## V. CUESTIONARIO

### V.1 Punteros inteligentes en C++

Los **punteros inteligentes** (*smart pointers*) fueron introducidos en C++11 mediante la biblioteca `<memory>`. A diferencia de los punteros ordinarios, gestionan automáticamente la propiedad del recurso mediante el principio RAII (*Resource Acquisition Is Initialization*), liberando el objeto en su destructor y admitiendo semántica de copia y de movimiento.

| Tipo | Propiedad | Escenario de uso típico |
|---|---|---|
| `std::unique_ptr` | Exclusiva (no copiable) | Propietario único del recurso; representa la ausencia de propiedad compartida. Es el valor por defecto siempre que sea posible. |
| `std::shared_ptr` | Compartida (contador atómico) | Varios componentes comparten la propiedad; el objeto se destruye cuando el último `shared_ptr` se libera. |
| `std::weak_ptr` | No propietaria (observador) | Rompe los ciclos de referencia de `shared_ptr` y permite observar la existencia del objeto sin impedir su destrucción. |

**Ejemplo 1:** `unique_ptr` **para propiedad exclusiva**

**Listado 12:** Propiedad exclusiva con unique$_ptr(ejemploilustrativo)$

```
// El ejercicio 1 construye el objeto con new + punteros crudos:
Arquitecto* arq = new Arquitecto("Lucia", "Fernandez Ramirez", 41,
                                 "DNI 43829175", 15, 3500.0,
                                 "Arq. Sustentable", "SIC-2011-4478", 28);
// El mismo objeto, con propiedad gestionada y liberacion automatica:
{
    std::unique_ptr<Persona> p = std::make_unique<Arquitecto>(
        "Lucia", "Fernandez Ramirez", 41, "DNI 43829175", 15, 3500.0,
        "Arq. Sustentable", "SIC-2011-4478", 28);
    p->presentarHabilidad();          // operator-> provisto por unique_ptr
}   // al salir del bloque se invocan ~Arquitecto() y ~Persona():
    // no hace falta delete y no puede producirse una fuga.
```

**Ejemplo 2:** `shared_ptr` **y** `weak_ptr` **para ciclos**

**Listado 13:** Rompimiento de ciclos de referencia con weak$_ptr(situacionhipotetica)$

```
// Situacion hipotetica: un nodo que guardara un enlace a su PROPIO padre.
// El ejercicio 3 no la necesita porque la jerarquia es un ARBOL
// (cada nodo es dueno exclusivo de sus dos hijos, sin retorno al padre).
std::shared_ptr<Nodo> raiz = std::make_shared<NodoOperacion>(
    '+', std::make_shared<Hoja>(3), std::make_shared<Hoja>(4));

```

## SOLUCIÓN Y RESULTADOS

```
// Si el padre guardara al hijo con shared_ptr Y el hijo al padre con
// shared_ptr, se formaria un CICLO: el contador de referencias nunca
// llega a cero, el destructor nunca se invoca y la memoria se pierde.
// La solucion es que UNO de los dos enlaces sea debil:
std::weak_ptr<Nodo> padreDelHijo = raiz;   // observa sin retener
```

En el Árbol de Expresiones del ejercicio 3, cada `NodoOperacion` es propietario exclusivo de sus dos hijos, de modo que la jerarquía se corresponde exactamente con un árbol de `unique_ptr`: al destruir la raíz se libera en cascada todo el subárbol, en $O(n)$ para $n$ nodos.

### V.2 Sobrecarga versus polimorfismo

La **sobrecarga** (*overloading*) y el **polimorfismo** son mecanismos distintos que comparten la misma apariencia superficial: permiten invocar una misma operación con resultados diferentes según el contexto. La diferencia radica en el **momento de resolución** y en el **tipo de relación** que subyace.

| Criterio | Sobrecarga | Polimorfismo |
|---|---|---|
| **Momento de resolución** | Enlace estático (tiempo de compilación). | Enlace dinámico (tiempo de ejecución). |
| **Criterio de selección** | Firma: número y tipo de los parámetros. | Tipo dinámico real del objeto receptor. |
| **Relación entre clases** | Ninguna: funciones independientes en el mismo ámbito. | Relación de herencia *es un* entre clases. |
| **Tabla de despacho** | Ninguna: se resuelve por el compilador. | *vtable* y `vptr` en cada objeto. |
| **Extensibilidad** | Limitada: cada firma nueva obliga a recompilar el cliente. | Abierta: una nueva derivada se integra sin tocar el cliente. |

**Ejemplo de sobrecarga (mismo nombre, distintas firmas):**

**Listado 14:** Sobrecarga de operadores resuelta estaticamente (ejemplo ilustrativo)

```
struct Vector2D {
    double x, y;
    Vector2D operator+(const Vector2D& v) const {   // sobrecarga de +
        return {x + v.x, y + v.y};
    }
    double operator*(double k) const {              // sobrecarga de *
        return x * k + y * k;   // firma distinta: no es polimorfismo
    }
};
// El compilador decide segun los ARGUMENTOS:
//   v1 + v2   ->  operator+(const Vector2D&)
//   v1 * 3.0  ->  operator*(double)
```

## SOLUCIÓN Y RESULTADOS

**Ejemplo de polimorfismo (misma firma, distinto cuerpo por clase):**

**Listado 15:** Polimorfismo resuelto dinamicamente mediante virtual (ejemplo ilustrativo)

```cpp
class Figura {
public:
    virtual double area() const = 0;    // una sola firma; decide el objeto
};
class Circulo : public Figura {
    double r;
public:
    double area() const override { return 3.14159 * r * r; }
};
class Cuadrado : public Figura {
    double l;
public:
    double area() const override { return l * l; }
};
std::vector<Figura*> figs = { new Circulo(2.0), new Cuadrado(3.0) };
for (Figura* f : figs)
    std::cout << f->area() << "\n";  // 12.566 y 9.0: despacho dinamico
```

En síntesis, la **sobrecarga** resuelve *qué* función invocar en función de los *argumentos*, mientras que el **polimorfismo** resuelve *qué* implementación ejecutar en función del *objeto receptor*. Ambos son complementarios y conviven en un mismo programa: el ejercicio 1 aplica sobrecarga en las funciones auxiliares de presentación y polimorfismo en `calcularSalarioMensual()`, redefinido por cada oficio.

## VI. CONCLUSIONS

1. La herencia constituye el mecanismo fundamental de reutilización y clasificación jerárquica en C++, permitiendo modelar relaciones *es un* del dominio y declarando los atributos y métodos comunes una única vez en la clase base.
2. Los niveles de acceso aplicados al acto de heredar (`public`, `protected`, `private`) van más allá de la visibilidad de los miembros: controlan si la relación de herencia se hace parte de la interfaz pública de la clase derivada.
3. El polimorfismo, implementado mediante funciones virtuales, transforma una interfaz única en múltiples comportamientos en tiempo de ejecución, y la declaración de un destructor virtual en la clase base resulta imprescindible para la liberación segura de memoria dinámica.
4. La herencia múltiple permite componer fenotipos ortogonales mediante mixins, pero exige la herencia virtual para resolver el problema del diamante y la identificación de tipos en tiempo de ejecución mediante RTTI (`dynamic_cast`) para consultar la presencia de un comportamiento concreto.
5. El Árbol de Expresiones Binario demuestra cómo la combinación de herencia, polimorfismo y punteros implementa un evaluador de expresiones que respeta la precedencia aritmética, con la idoneidad de la recursión para la evaluación y la liberación del árbol.

## SOLUCIÓN Y RESULTADOS

6. Los punteros inteligentes consolidan el modelo de propiedad de C++ mediante RAII, eliminando la necesidad de `delete` manual y previniendo tanto las fugas de memoria como los punteros colgantes, con `unique_ptr` como opción preferente por defecto.
7. La distinción entre sobrecarga y polimorfismo resulta fundamental: la primera opera mediante enlace estático sobre la firma de los argumentos, mientras que la segunda actúa mediante enlace dinámico sobre el tipo real del objeto; ambos mecanismos son complementarios en el diseño de sistemas-orientados a objetos.

## RETROALIMENTACIÓN GENERAL

## REFERENCIAS Y BIBLIOGRAFÍA

[1] Universidad Nacional de San Agustín. (2026). *Guía de Práctica de Laboratorio 03: C++ Herencia, Polimorfismo, Qt*. Escuela Profesional de Ingeniería de Sistemas. Código: GUIA-PRLD-001. Docente: Mg. Ing. Rene Alonso Nieto Valencia.

[2] Stroustrup, B. (2013). *The C++ Programming Language* (4.ª ed.). Addison-Wesley Professional.

[3] Deitel, P., y Deitel, H. (2017). *C++ How to Program* (10.ª ed.). Pearson.

[4] Subramanyam, A., y Siewiorek, J. (2008). *Reliability and Fault Tolerance in the Design of Computer Systems*. Wiley-IEEE Press.

[5] cppreference.com. (2024). *std::unique_ptr, std::shared_ptr, std::weak_ptr*. `https://en.cppreference.com/w/cpp/memory`